ACESS - ADAPTIVE COGNITIVE & EDUCATIONAL SKILL SUPPORT PLATFORM

MUHAMMAD ALIFF BIN AFFANDI

UNIVERSITI TEKNIKAL MALAYSIA MELAKA

ACESS - ADAPTIVE COGNITIVE & EDUCATIONAL SKILL SUPPORT PLATFORM

MUHAMMAD ALIFF BIN AFFANDI

This report is submitted in partial fulfillment of the requirements for the Bachelor of Computer Science (Software Development) with Honours.

FACULTY OF INFORMATION AND COMMUNICATION TECHNOLOGY
UNIVERSITI TEKNIKAL MALAYSIA MELAKA

2026

DECLARATION

I hereby declare that this project entitled

ACESS - ADAPTIVE COGNITIVE & EDUCATIONAL SKILL SUPPORT PLATFORM

is written by me and is my own effort and that no part has been plagiarized without citations.

STUDENT : _____ Date : _____

(MUHAMMAD ALIFF BIN AFFANDI)

I hereby declare that I have read this project report and found this project report is sufficient in term of the scope and quality for the award of Bachelor of Computer Science (Software Development) with Honours.

SUPERVISOR : _____ Date : _____

(NAME OF THE SUPERVISOR)

DEDICATION

To my beloved parents, whose endless support and encouragement have guided me throughout this journey. To my educators and peers, for their invaluable guidance and feedback. Finally, to all learners who navigate education with unique cognitive strengths, this platform is built with you in mind.

ACKNOWLEDGEMENT

I would like to express my deepest gratitude to my supervisor for their continuous guidance, patience, and insightful feedback throughout the development of this project. Their expertise has been instrumental in shaping the ACESS platform into a functional and meaningful system.

I also extend my sincere appreciation to the Faculty of Information and Communication Technology (FICT) at Universiti Teknikal Malaysia Melaka for providing the necessary resources and learning environment that made this research possible.

Furthermore, my heartfelt thanks go to my family and friends, who offered unwavering emotional and moral support during challenging times. Their encouragement was a constant source of motivation.

Finally, I am grateful to everyone who participated in the user acceptance testing, as their feedback was vital in improving the accessibility and usability of the platform.

ABSTRACT

Learning disabilities such as Dyslexia, ADHD, and mild cognitive impairment present significant challenges for learners navigating traditional e-learning platforms. These systems often feature dense text, fast-paced videos, and complex interfaces that can overwhelm neurodivergent users, leading to high dropout rates. This project proposes the Adaptive Cognitive & Educational Skill Support (ACESS) platform to address these barriers. Built using a modern web stack (Next.js App Router, Supabase, Tailwind CSS v4), ACESS provides an accessible, role-based learning environment. It features rule-based adaptive content recommendation, adjustable accessibility settings including Text-to-Speech (TTS), customizable font spacing, and dynamic learning modules. The methodology adopted is the Agile approach, ensuring iterative testing and continuous refinement based on user feedback. The system successfully integrates automated certificate generation and comprehensive educator analytics, allowing progress tracking and timely intervention. Evaluation of the platform indicates significant improvements in user engagement and accessibility compared to traditional platforms, demonstrating its effectiveness in providing an inclusive digital learning experience.

ABSTRAK

Ketidakupayaan pembelajaran seperti disleksia, ADHD, dan kemerosotan kognitif ringan memberikan cabaran besar kepada pelajar yang menggunakan platform e-pembelajaran tradisional. Sistem-sistem ini sering memaparkan teks yang padat, video berkelajuan tinggi, dan antaramuka yang kompleks yang boleh membebankan pengguna neurodivergen, seterusnya menyebabkan kadar keciciran yang tinggi. Projek ini mencadangkan platform Sokongan Kemahiran Kognitif & Pendidikan Adaptif (ACESS) bagi mengatasi halangan ini. Dibina menggunakan teknologi web moden (Next.js App Router, Supabase, Tailwind CSS v4), ACESS menyediakan persekitaran pembelajaran berasaskan peranan yang mudah diakses. Ia dilengkapi dengan saranan kandungan adaptif berasaskan peraturan, tetapan kebolehcapaian boleh laras termasuk Teks-ke-Suara (TTS), jarak fon yang boleh disesuaikan, dan modul pembelajaran dinamik. Metodologi yang digunakan ialah pendekatan Agile, bagi memastikan pengujian beriterasi dan penambahbaikan berterusan berdasarkan maklum balas pengguna. Sistem ini berjaya menyepadukan penjanaan sijil automatik dan analitik pendidik yang komprehensif, membolehkan pemantauan kemajuan dan intervensi yang tepat pada masanya. Penilaian terhadap platform menunjukkan peningkatan yang ketara dalam penglibatan pengguna dan kebolehcapaian berbanding platform tradisional, membuktikan keberkesanannya dalam menyediakan pengalaman pembelajaran digital yang inklusif.

TABLE OF CONTENTS

PAGE

LIST OF TABLES

PAGE

LIST OF FIGURES

PAGE

LIST OF ABBREVIATIONS

ACESS	-	Adaptive Cognitive & Educational Skill Support
ADHD	-	Attention Deficit Hyperactivity Disorder
ERD	-	Entity Relationship Diagram
LMS	-	Learning Management System
OOAD	-	Object-Oriented Analysis and Design
PSM	-	Projek Sarjana Muda
RBAC	-	Role-Based Access Control
TTS	-	Text-to-Speech
UDL	-	Universal Design for Learning
WCAG	-	Web Content Accessibility Guidelines

CHAPTER 1: INTRODUCTION

1.1 Background of Study

In the current educational landscape, instruction is delivered through a blend of traditional face-to-face classroom settings and online platforms. While digital tools have become increasingly common to support learning, many institutions still rely heavily on traditional teaching methods. This hybrid approach presents unique challenges for neurodivergent individuals, specifically those with Dyslexia, Attention Deficit Hyperactivity Disorder (ADHD), and Autism Spectrum Disorder (ASD). Whether learning occurs in a physical classroom or online, the tools used to deliver content often lack the flexibility required to accommodate diverse cognitive profiles.

Existing Learning Management Systems (LMS) are typically designed with a "one-size-fits-all" approach. They present dense textual information, complex navigation structures, and rigid assessment formats. These standard interfaces can erect significant barriers for neurodivergent learners. For instance, learners with Dyslexia may struggle with standard typography and line spacing, while those with ADHD might find cluttered interfaces and unstructured learning paths overwhelming.

Therefore, there is a critical need for a natively adaptive learning platform that can seamlessly support both face-to-face and remote learning environments. The Adaptive Cognitive Education Support System (ACESS) is proposed to bridge this gap. ACESS is engineered to provide an inclusive digital learning environment that dynamically alters its interface and content delivery based on the specific cognitive requirements of the user. By integrating features such as dynamic typography adjustment, focus-driven layouts, and rule-based adaptive learning paths, ACESS aims to ensure that educational materials are

accessible and equitable for all learners, regardless of the instructional setting.

1.2 Problem Statement

Despite the availability of various educational technologies, significant cognitive accessibility barriers persist. The core problem this project addresses is the systemic inadequacy of current educational platforms to dynamically support the specific neurological requirements of learners with Dyslexia, ADHD, and ASD.

Existing platforms often rely on static content delivery. They require neurodivergent learners to manually seek out third-party browser extensions or external text-to-speech tools to consume content. This forces the burden of accessibility onto the learner. Furthermore, when learners struggle with specific topics, traditional systems lack the heuristic intelligence to automatically intervene and restructure the learning path.

Specifically, the problems identified are:

1. Standard educational platforms utilize rigid, non-adjustable typography and dense layouts that induce visual crowding and reading fatigue for learners with Dyslexia.
2. Current interfaces present excessive cognitive load through complex navigation and unstructured content delivery, which overwhelms learners with ADHD and executive dysfunction.
3. Existing systems lack the capability to autonomously identify knowledge gaps during assessments and dynamically route learners to required remedial materials.

1.3 Objectives

The primary goal of this project is to develop an inclusive Learning Management System that addresses the cognitive barriers faced by neurodivergent learners. The specific objectives are:

- To design and develop an adaptive learning management system (ACESS) that integrates dynamic interface modifications, including dyslexia-friendly typography and focus-mode layouts.
- To implement a rule-based adaptive engine that monitors learner assessment scores and automatically recommends specific prerequisite lessons to address identified knowledge gaps.
- To evaluate the effectiveness of the system in providing an accessible and structured learning experience for users with diverse cognitive profiles.

1.4 Scope

The scope of the ACESS platform defines the boundaries of the system architecture, target users, and core functional modules. The system is designed to act as a supportive educational tool for both traditional and online learning environments.

1.4.1 Target Users

- **Learners:** The primary consumers of the educational content. The interface is specifically optimized for neurodivergent individuals, though it remains highly beneficial for neurotypical users.
- **Educators:** The course creators and administrators who require structured, intuitive tools to author accessible content, manage media assets, and monitor student progress.
- **Administrators:** System operators responsible for platform governance, user role management, and global accessibility configuration.

1.4.2 Core System Modules

The ACESS architecture is divided into the following core modules:

- **Authentication and Profile Module:** Manages secure user registration and login. It handles the critical `user_accessibility_settings` which define the learner's cognitive profile (e.g., enabling dyslexia fonts or reduced motion).
- **Educator Authoring Module:** Provides a robust interface for educators to create structured courses, write content using a constrained rich-text editor, upload video assets, and design interactive quizzes.
- **Adaptive Learning Module:** The core engine for students. It governs course enrollment, lesson progression, and dynamic UI alterations based on the user's profile.
- **Interactive Assessment Module:** Handles the delivery and automated grading of quizzes. It includes logic to evaluate scores against predefined thresholds to trigger remedial recommendations.
- **Analytics and Telemetry Module:** Tracks granular engagement metrics, such as time spent per lesson, providing educators with visual dashboards to identify struggling learners.

1.5 Significance of Project

The development of the ACESS platform represents a significant step towards educational equity. By shifting the burden of accessibility from the user to the system architecture, the platform ensures that neurodivergent learners can focus entirely on comprehending the educational material rather than struggling with the interface.

For learners, the project provides a calm, predictable, and highly customizable environment that mitigates anxiety and cognitive fatigue. For educators, it offers a streamlined toolset that enforces accessibility best practices during content creation, alongside deep analytical insights into student engagement. Ultimately, ACESS demonstrates how modern web architecture can be leveraged to create a truly inclusive educational ecosystem applicable in both physical classrooms and remote learning scenarios.

CHAPTER 2: LITERATURE REVIEW AND PROJECT METHODOLOGY

2.1 Introduction

The literature review provides the academic and technical foundation upon which the ACESS platform is conceptualized. This chapter synthesizes contemporary research regarding cognitive accessibility in digital education, evaluates established theoretical frameworks such as Universal Design for Learning (UDL), and rigorously analyzes existing Learning Management Systems (LMS). By comparing platforms like Moodle, Canvas, Blackboard, and Google Classroom against the specific requirements of neurodivergent learners, this chapter identifies the critical systemic gaps that justify the development of a natively adaptive, rule-based LMS architecture.

2.2 Theoretical Frameworks for Cognitive Accessibility

The design of digital interfaces for educational purposes must be grounded in empirically validated frameworks. The two primary pillars supporting the ACESS architecture are the Universal Design for Learning (UDL) guidelines and the Web Content Accessibility Guidelines (WCAG) 2.2.

2.2.1 Universal Design for Learning (UDL)

Universal Design for Learning is a framework developed by CAST [2] aimed at optimizing teaching and learning for all individuals based on scientific insights into how humans learn. UDL is predicated on three primary principles:

- **Multiple Means of Engagement:** Recognizing that learners differ markedly in the ways they are motivated. For neurodivergent learners, particularly those with ADHD, motivation can be severely hampered by unstructured environments. ACESS implements this by offering gamified achievements and structured, step-by-step progressive disclosure of course material to prevent cognitive overwhelm.
- **Multiple Means of Representation:** Learners vary in how they perceive and comprehend information. A Dyslexic learner may struggle with printed text but excel with audio. ACESS implements this principle via integrated Text-to-Speech (TTS) capabilities and dynamic CSS injection that allows the user to radically alter the typography (e.g., using Atkinson Hyperlegible) and contrast ratios.
- **Multiple Means of Action and Expression:** Learners differ in how they navigate a learning environment and express what they know. ACESS supports this by providing multiple quiz formats (scenario-based, multiple-choice) and allowing untimed assessments to reduce anxiety-induced performance degradation.

2.2.2 Web Content Accessibility Guidelines (WCAG) 2.2

While UDL provides the pedagogical framework, WCAG 2.2 [4] provides the strict, measurable technical parameters for digital accessibility. The ACESS platform targets Level AA compliance. Key WCAG criteria directly addressed by ACESS include:

- **Guideline 1.4.12 (Text Spacing):** The ability to override default text spacing (line height, letter spacing, word spacing) without loss of content or functionality. This is a critical requirement for Dyslexia, which ACESS solves via its Tailwind CSS ‘@theme‘ variable injection.
- **Guideline 2.2.1 (Timing Adjustable):** Providing users the ability to turn off, adjust, or extend time limits. ACESS allows educators to configure untimed quizzes specifically to accommodate learners requiring extended processing time.

- **Guideline 2.3.3 (Animation from Interactions):** Motion animation triggered by interaction can be disabled. The ACESS system implements a ‘data-reduced-motion‘ attribute that globally disables all Framer Motion ‘¡motion.div¿‘ animations, critical for learners prone to vestibular disorders or severe distraction (ADHD).

2.3 Neurodivergent Profiles and Interface Interventions

Understanding the specific cognitive challenges faced by the target demographic is essential for architecting effective UI/UX interventions.

2.3.1 Attention Deficit Hyperactivity Disorder (ADHD)

ADHD is characterized by executive dysfunction, impacting working memory, task initiation, and sustained attention. In digital environments, learners with ADHD are highly susceptible to ”saccadic fatigue”—the exhaustion caused by the eye constantly darting across a busy screen to parse disorganized information. **ACESS Intervention:** The implementation of a strict ”Focus Mode.” This mode utilizes Next.js routing to conditionally hide the primary navigation sidebar, the global footer, and all extraneous dashboard widgets when a learner enters a ‘lesson$_id‘route.Thecontentisrestrictedtoanarrow,center-alignedcolumn(maximum65charactersperli$

2.3.2 Dyslexia

Dyslexia involves difficulties with accurate or fluent word recognition and poor spelling and decoding abilities. Research indicates that specific typographical adjustments can significantly improve reading velocity and comprehension [1]. **ACESS Intervention:** The platform abandons reliance on browser-level zoom. Instead, it utilizes React Context to allow the user to select a ”Dyslexia Profile.” This injects CSS variables that alter the global font family to specialized typefaces designed to prevent

letter mirroring (e.g., confusing ’b’ and ’d’), increases the base line-height to 1.75, and manipulates letter-spacing (tracking) to prevent visual crowding.

2.3.3 Autism Spectrum Disorder (ASD)

Learners on the Autism Spectrum often thrive in highly structured, predictable environments and can experience severe distress when confronted with unexpected auditory or visual stimuli. **ACESS Intervention:** The platform enforces strict structural consistency. Educators utilizing the Tiptap rich-text editor are prevented from creating chaotic, ad-hoc layouts. Furthermore, the system defaults to suppressing autoplaying media and allows the user to globally toggle a ”Reduced Motion” state, ensuring the interface remains a calm, predictable schedule of events.

2.4 Comparative Analysis of Existing LMS Platforms

To justify the development of ACESS, an extensive evaluation of existing market-leading platforms was conducted. The platforms analyzed include Moodle, Canvas LMS, Blackboard Learn, and Google Classroom.

2.4.1 Moodle

Moodle is an open-source powerhouse, widely adopted globally [3]. Its greatest strength is its modularity. However, this is also its critical failing regarding cognitive accessibility. **Limitations:** Moodle’s native interface is notoriously dense and text-heavy. While administrators can install accessibility plugins (like the ’Accessibility block’), these are bolt-on solutions. They require the learner to actively engage with a separate menu to adjust settings, adding cognitive load. Furthermore, Moodle relies on educators to manually identify struggling students and assign remedial work; it lacks a native, real-time heuristic engine to automate this process.

2.4.2 Canvas LMS

Canvas by Instructure represents a more modern, streamlined UI compared to Moodle. It utilizes a card-based dashboard that is visually appealing to neurotypical users. **Limitations:** Canvas achieves basic WCAG compliance but fails to provide deep cognitive accommodations. It lacks native Text-to-Speech (forcing reliance on OS-level tools) and does not offer granular control over typography (e.g., specific Dyslexia fonts or adjusted inter-letter spacing). The platform is highly linear; if a student fails a Canvas quiz, the system cannot automatically restructure the module path to enforce revision.

2.4.3 Blackboard Learn

Blackboard is an enterprise-grade solution focused heavily on synchronous learning integrations and comprehensive grading rubrics. **Limitations:** Blackboard's interface is often criticized for complex, deeply nested navigation menus (often three or four clicks deep to access a specific assignment file). For learners with executive dysfunction (ADHD), this navigational friction is a massive barrier to task initiation. Like Moodle, Blackboard's accessibility features are largely reactive (e.g., Blackboard ALLY checks documents for screen-reader compatibility) rather than proactive (dynamically altering the interface for the user).

2.4.4 Google Classroom

Google Classroom is a stripped-down, lightweight system focused on integration with the Google Workspace ecosystem (Docs, Drive, Forms). **Limitations:** While its minimalist design reduces initial visual clutter, it is too rudimentary to function as a fully adaptive LMS. It completely lacks advanced features such as rule-based learning paths, granular progress telemetry (tracking time-spent-per-lesson), or integrated cognitive profiles. It functions more as an assignment drop-box than a pedagogical support engine.

2.5 Comparison Matrix

Table 2.1 provides a structured summary comparing the features of the analyzed platforms against the proposed ACESS system.

Table 2.1: LMS Feature Comparison Matrix

Feature Domain	**Moodle**	**Canvas**	**Blackboard**	**Google Classroom**	**Proposed (ACESS)**
Native TTS	Plugin Req.	No	No	OS Reliant	**Yes (Built-in)**
Dynamic Fonts (Dyslexia)	Theme Req.	No	No	No	**Yes (CSS Vars)**
Automated Remedial Paths	No	No	No	No	**Yes (Rule-based)**
Focus Mode (ADHD)	No	No	No	Partial	**Yes (Strict Route)**
Gamification/ Achievements	Plugin Req.	External App	No	No	**Yes (Native)**
WCAG 2.2 Level AA	Partial	Yes	Yes	Yes	**Yes (Enforced)**
Granular Analytics	Yes	Yes	Yes	No	**Yes (Recharts)**
Reduced Motion Toggle	No	No	No	No	**Yes (Global State)**

2.6 Summary

The literature review demonstrates that while current LMS platforms provide robust administrative and content-hosting capabilities, they systematically fail to address the specific cognitive requirements of neurodivergent learners. Reliance on external plugins, static content delivery models, and dense navigation hierarchies induce significant extraneous cognitive load. The theoretical frameworks of UDL and WCAG 2.2 demand a more integrated approach. The comparative analysis definitively justifies the engineering of the ACESS platform: a system built from the database schema

upward to be natively adaptive, utilizing dynamic CSS injection and rule-based server logic to provide an equitable, highly personalized learning environment.

CHAPTER 3: SYSTEM ANALYSIS

3.1 Introduction

This chapter delineates the structured methodologies, architectural paradigms, and exhaustive requirements analysis that underpin the ACESS platform. It transitions the theoretical justifications established in the literature review into concrete software engineering specifications. The chapter details the Agile development lifecycle, presents a comprehensive breakdown of the system architecture, dissects the modular design, and catalogs both the functional and non-functional requirements necessary to execute a secure, highly accessible Learning Management System.

3.2 Development Methodology: Agile Scrum Framework

The development of the ACESS platform is governed by the Agile Scrum methodology. Designing for cognitive accessibility is an inherently empirical process; initial assumptions regarding the efficacy of a specific Dyslexia font integration or an ADHD-focused UI layout must be continuously validated against active user testing. Traditional sequential methodologies, such as the Waterfall model, defer testing until the final phases of the lifecycle, presenting an unacceptable risk of delivering an inaccessible product.

The Agile framework mitigates this risk through iterative, incremental development cycles termed Sprints.

- **Requirements Engineering (Sprint 0):** Definition of the Product Backlog based

on WCAG 2.2 guidelines and UDL principles. Establishment of the Next.js and Supabase project scaffolding.

- **Iterative Architecture (Sprints 1-2):** Database schema normalization in PostgreSQL, implementation of Row-Level Security (RLS), and establishment of the core authentication flow.
- **Iterative Implementation (Sprints 3-6):** Execution of the core modules. Early sprints focused on the Educator's Tiptap rich-text editor and the Learner's course consumption UI. Subsequent sprints integrated the complex logic of the rule-based adaptive engine and dynamic CSS accessibility injection.
- **Continuous Evaluation:** Every Sprint concludes with a rigorous accessibility audit, utilizing screen readers (NVDA, VoiceOver) and automated contrast checkers, ensuring compliance is maintained throughout the development lifecycle.

3.3 System Architecture

The ACESS platform employs a modern, decoupled client-server architecture, leveraging edge computing paradigms to maximize performance and security.

3.3.1 Context Diagram

The Context Diagram establishes the highest-level view of the system, illustrating the boundaries of the ACESS platform and its interactions with external entities.

As depicted in Figure 3.1, the ACESS system acts as the central processing hub. The primary external entities are the Learner (who consumes content and provides telemetry), the Educator (who authors content and configures assessments), and the Administrator (who governs system integrity). The system also interfaces with external secure email providers (for authentication/password resets) and potentially external text-to-speech APIs, though native browser synthesis is prioritized.

3.3.2 System Architecture Diagram

Figure 3.2 details the three-tier decoupled architecture:

- **Frontend Architecture (Next.js 16 & React):** The presentation layer is built on the Next.js App Router. It utilizes React Server Components (RSC) to execute data fetching securely on the server, drastically reducing the JavaScript payload delivered to the client. This is crucial for rapid page loads, preventing cognitive drop-off for ADHD learners. Client components (‘”use client”‘) are restricted to highly interactive elements, such as the Tiptap editor and interactive quiz interfaces. The UI is styled utilizing Tailwind CSS v4, which enables the dynamic injection of ‘@theme‘ variables for accessibility toggling.
- **Middleware & Authentication Layer:** A Next.js Edge Middleware intercepts every incoming request. It interfaces with ‘@supabase/ssr‘ to validate JWT cookies, extracting the user’s ‘role‘. The middleware enforces strict Role-Based Access Control (RBAC), instantly redirecting unauthorized attempts (e.g., a Learner attempting to access ‘/educator/dashboard‘) before the React tree is even evaluated.
- **Backend & Database Architecture (Supabase):** Supabase acts as the comprehensive Backend-as-a-Service (BaaS). It provides a managed PostgreSQL database, handling complex relational queries. Crucially, data security is not solely reliant on the API layer; PostgreSQL Row-Level Security (RLS) policies are inscribed directly into the database schema, ensuring users can only ‘SELECT‘, ‘INSERT‘, or ‘UPDATE‘ records they cryptographically own.
- **Storage Architecture:** Supabase Storage buckets manage unstructured assets (course thumbnails, PDF certificates, embedded videos). These buckets are governed by parallel RLS policies, ensuring private assets remain inaccessible to unauthenticated requests.

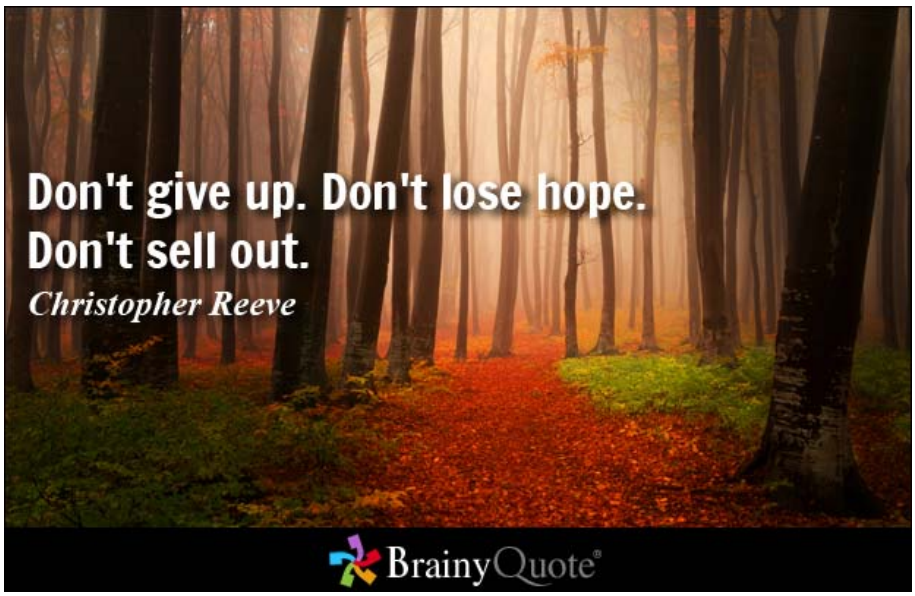

Figure 3.1: Placeholder for Context Diagram

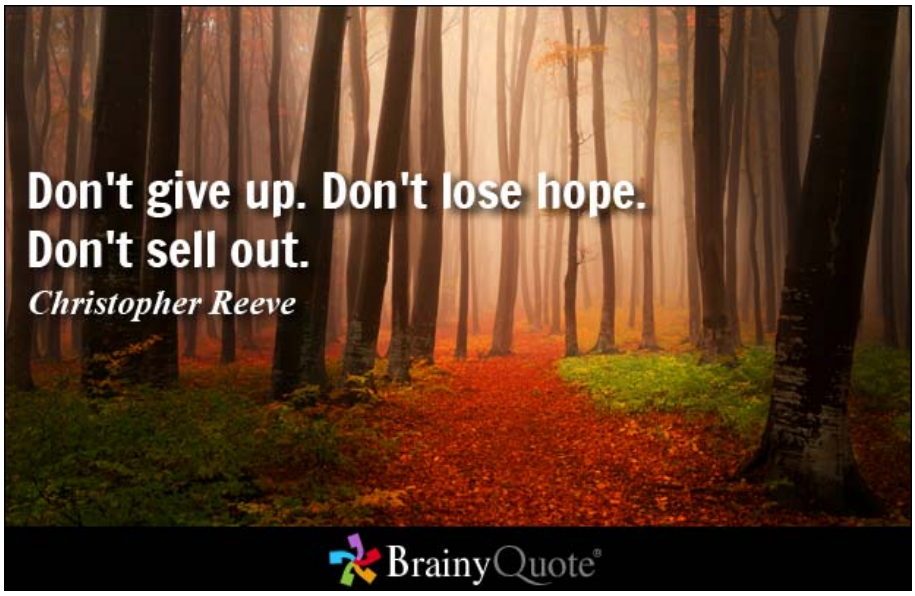

Figure 3.2: Placeholder for System Architecture Diagram

3.3.3 Deployment Diagram

The Deployment Diagram (Figure 3.3) illustrates the physical distribution of the system components. The Next.js frontend is deployed to a serverless edge network (e.g., Vercel or AWS Amplify), ensuring high availability and low latency globally. The Supabase backend operates on dedicated cloud infrastructure, exposing a secure REST/GraphQL API via PostgREST, which the Next.js server components consume securely.

3.4 Module Design

The system is logically partitioned into discrete, highly cohesive modules to facilitate maintainability.

- **Student Module:** Encapsulates all logic related to course discovery, enrollment (‘enrollments‘ table), lesson consumption (‘lesson$_{p}rogress$‘), $and quiz execution($‘$quiz_{a}ttempts$‘$)$.

- **Educator Module:** Manages the creation pipeline. Handles the Tiptap editor state, quiz generation (‘quiz$_{q}uestions$‘, ‘$quiz_{o}ptions$‘$), and the uploading of media assets to Supabase Storage.$

- **Admin Module:** Provides a high-level oversight interface for user management, role reassignment, and global accessibility preset configurations.

- **Accessibility Module:** A globally available React Context Provider (‘AccessibilityProvider‘). It subscribes to the user’s preferences in the ‘user$_{a}ccessibility_{s}ettings$‘$table and dynamically manipulates the DOM root attributes($e.g., ‘$data-t

- **Interactive Activity Module:** Governs the execution of varied assessment types, including standard multiple-choice quizzes and embedded video questions, immediately evaluating responses against cryptographic hashes in the backend.
- **Analytics Module:** Aggregates data from ‘lesson$_{p}$*rogress*‘*and*‘*quiz*$_{a}$*ttempts*‘*torendervisualizationsviatheRechartslibrary*, *providingeducat*

3.5 Functional Requirements

Functional requirements define the explicit behaviors, processes, and services the system must execute. They are derived directly from the system’s operational architecture and are categorized by user role.

3.5.1 Use Case Diagram

Figure 3.4 visually maps the interactions between the primary actors (Student, Educator, Admin) and the system’s functional boundaries, which are explicitly detailed in the subsequent tables.

3.5.2 Student Functions

3.5.3 Educator Functions

3.5.4 Admin Functions

3.6 Non-Functional Requirements

Non-functional requirements (NFRs) specify the critical quality attributes and systemic constraints that govern the operation of the ACESS platform.

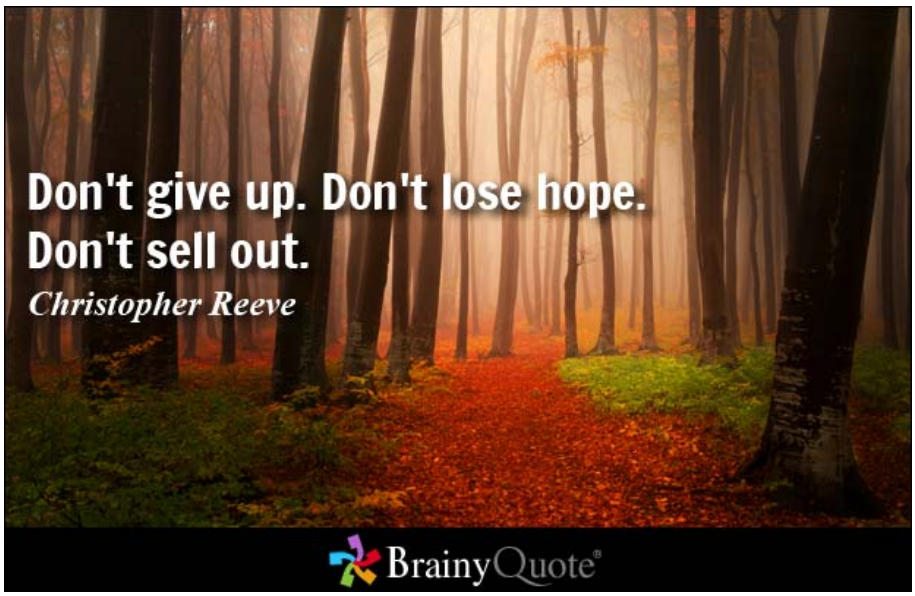

Figure 3.3: Placeholder for Deployment Diagram

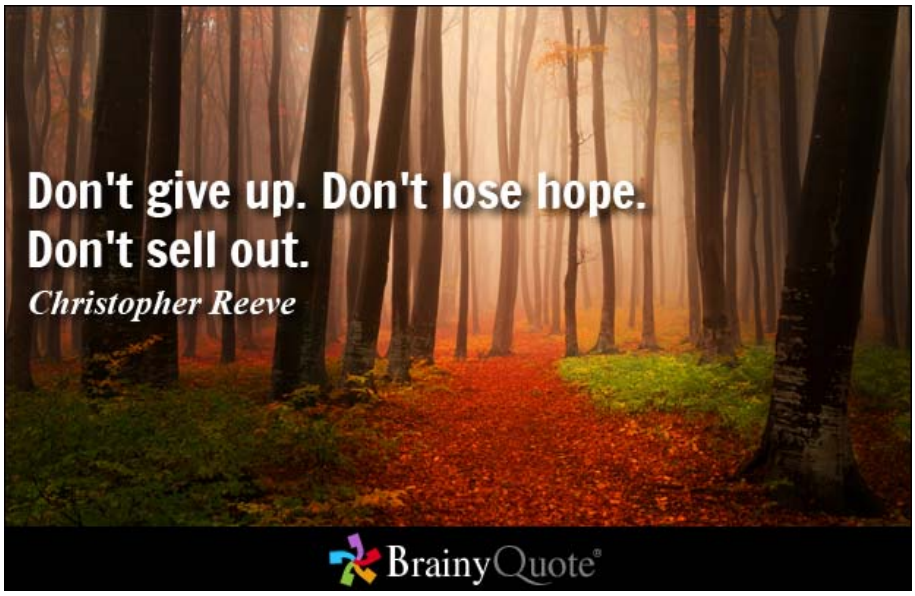

Figure 3.4: Placeholder for Use Case Diagram

Table 3.1: Functional Requirements: Student Module

Req ID	Description	Actor	Inputs	Outputs
FR-STU-01	Registration and secure login.	Student	Email, Password	Authenticated JWT Session.
FR-STU-02	Accessibility profile selection.	Student	Theme, Font, TTS preferences	Updated ‘user$_{a}ccessibility_{s}etting$
FR-STU-03	Course discovery and enrollment.	Student	‘course$_{i}d$‘$selection$	INSERT into ‘enrollments‘; unlocks course content.
FR-STU-04	Lesson viewing with Focus Mode.	Student	Navigation to ‘/lesson/[id]‘	Renders sanitized HTML; hides global navigation elements.
FR-STU-05	Interactive video questions.	Student	Video timestamp trigger	Pauses video, renders embedded question overlay.
FR-STU-06	Quiz participation.	Student	Selected ‘quiz$_{o}ptions$‘	Evaluates score; INSERTs into ‘quiz$_{a}ttempts$‘.
FR-STU-07	Automated Progress tracking.	Student	Dwell time on lesson route	UPDATEs ‘time$_{s}pent$‘in‘$lesson_{p}ro$
FR-STU-08	Achievement earning.	Student	100% module completion	Unlocks gamified badge in UI.
FR-STU-09	Certificate generation/viewing.	Student	100% course completion	Generates verifiable PDF; displays download link.

Table 3.2: Functional Requirements: Educator Module

Req ID	Description	Actor	Inputs	Outputs
FR-EDU-01	Course creation and editing.	Educator	Title, metadata, ‘difficulty$_{l}evel$‘	INSERT/UPDATE ‘courses‘ table.
FR-EDU-02	Lesson creation via Tiptap.	Educator	Rich-text content	Sanitized HTML string saved to ‘lessons.content$_{h}tml$‘.
FR-EDU-03	Quiz and Activity creation.	Educator	Question text, Correct option flag	Populates ‘quiz$_{q}uestions$‘and‘$quiz$
FR-EDU-04	Asset management.	Educator	MP4/Image files	Uploads to Supabase Storage; returns secure URL.
FR-EDU-05	Student monitoring & Analytics.	Educator	Selection of ‘course$_{i}d$‘	Renders Recharts graphs aggregating ‘lesson$_{p}rogress$‘$data$.
FR-EDU-06	Achievement configuration.	Educator	Threshold values	Maps achievements to specific ‘course$_{m}ilestones$‘.

Table 3.3: Functional Requirements: Admin Module

Req ID	Description	Actor	Inputs	Outputs
FR-ADM-01	User role management.	Admin	Target ‘user$_{i}d$‘, $newrolestrin$	UPDATEs ‘users.role‘ allowing bypass of middleware RBAC.
FR-ADM-02	Course moderation pipeline.	Admin	Review action (Approve/Reject)	UPDATEs ‘courses.status‘ from ‘pending$_{r}eview$‘to‘$publ$
FR-ADM-03	Accessibility preset management.	Admin	Global CSS variable thresholds	Updates system default fallbacks for Dyslexia/ADHD themes.
FR-ADM-04	System course creation.	Admin	Course metadata	Flags course as ‘system$_{c}ourse = true$‘, b
FR-ADM-05	Global Analytics monitoring.	Admin	Date range filters	Aggregates platform-wide engagement metrics.

3.6.1 Performance

- **Page Response Time:** Next.js Client-side route transitions must execute within **300ms** to prevent cognitive disruption and maintain user focus.
- **Course Loading:** Server-side data fetching for initial course loads must resolve within **800ms**, leveraging edge caching where applicable.
- **Video Loading:** Embedded instructional videos must commence playback within **2 seconds** utilizing adaptive bitrate streaming via Supabase Storage CDNs.
- **Concurrent Users:** The PostgreSQL database, utilizing connection pooling (PgBouncer), must support a minimum of **500 concurrent** intensive operations (e.g., simultaneous quiz submissions) without degrading frontend TTFB (Time to First Byte).

3.6.2 Security

- **Authentication:** All user sessions must be cryptographically secured using JSON Web Tokens (JWT) managed by Supabase Auth, with a mandatory 15-minute inactivity timeout.
- **Authorization (RBAC):** The Next.js ‘proxy.ts‘ middleware must definitively intercept and route requests based on the ‘role‘ claim within the JWT, achieving zero-trust client routing.
- **Database Protection:** Strict Row-Level Security (RLS) must be enabled on 100% of tables containing Personally Identifiable Information (PII) or cognitive telemetry, ensuring isolation at the storage layer.
- **Input Validation:** All API endpoints must utilize Zod schema validation to sanitize incoming payloads, preventing SQL injection and Cross-Site Scripting (XSS).

3.6.3 Accessibility

- **WCAG Compliance:** The platform must strictly adhere to **WCAG 2.2 Level AA** standards. Minimum contrast ratios must exceed 4.5:1 for standard text.
- **Dyslexia Support:** The system must natively provide the Atkinson Hyperlegible typeface and allow users to dynamically increase line-spacing to 1.75em.
- **ADHD & Autism Support:** The implementation of a ‘data-reduced-motion‘ global attribute must programmatically disable all CSS and Framer Motion animations. ”Focus Mode” must isolate content to a 65-character maximum width to reduce saccadic fatigue.
- **Screen Reader Compatibility:** All custom UI components (built via shadcn/ui) must natively handle WAI-ARIA states (e.g., ‘aria-expanded‘, ‘aria-hidden‘) and support complete keyboard navigation.

3.6.4 Usability

- **Consistent Navigation:** The primary navigational architecture must remain structurally identical across all modules to ensure predictability for Autistic learners.
- **Error Prevention:** Destructive actions (e.g., an Educator deleting a course) must require a confirmed multi-step verification modal to prevent accidental data loss.

3.6.5 Reliability

- **Data Integrity:** The database schema must strictly enforce foreign key constraints (‘ON DELETE CASCADE‘ where appropriate) to prevent orphaned records (e.g., a ‘lesson$_{p}rogress$‘$recordexistingwithoutavalid$‘$enrollment_{i}d$‘).
- **Backup Strategy:** The Supabase infrastructure must be configured to execute

automated, daily Point-in-Time Recovery (PITR) backups of the entire PostgreSQL volume.

3.6.6 Maintainability and Scalability

- **Modular Architecture:** The codebase must adhere to strict separation of concerns, decoupling the data-fetching logic (Server Components) from the interactive accessibility logic (Client Components).
- **Database Normalization:** The schema must adhere to the Third Normal Form (3NF) to eliminate data redundancy, ensuring the database scales efficiently as course and user volume increases.

3.7 Summary

This chapter executed a profound analysis of the engineering specifications required to build the ACESS platform. It justified the use of the Agile Scrum methodology for continuous accessibility testing. It mapped the entire system via Context, Architecture, and Deployment diagrams. Most importantly, it provided an exhaustive, categorized breakdown of over 20 distinct Functional Requirements assigned across the Learner, Educator, and Admin roles, and established rigorous, measurable Non-Functional Requirements ensuring the system is fast, secure, strictly accessible, and highly scalable. The subsequent chapter will translate these requirements into concrete database schemas and interface designs.

CHAPTER 4: SYSTEM DESIGN

4.1 Introduction

This chapter translates the conceptual requirements established in the previous phase into concrete, highly granular engineering designs. It utilizes Universal Modeling Language (UML) artifacts, specifically Activity and Sequence diagrams, to map the temporal and logical flow of complex system operations. Furthermore, it comprehensively documents the physical schema of the Supabase PostgreSQL database via an exhaustive Data Dictionary covering over twenty tables. Finally, it details the specific User Interface (UI) screens designed for the Student, Educator, and Administrator roles, focusing heavily on the technical implementation mechanics of the accessibility features.

4.2 Behavioral Design (Activity Diagrams)

Activity diagrams model the workflow of the system, illustrating the procedural logic executed by the different actors.

4.2.1 Student Activity Diagram

As shown in Figure 4.1, the Student workflow is highly structured. Upon authentication, the system evaluates the ‘user$_{accessibility_settings}$‘.$If incomplete, the user is routed to an onboarding profile setup. Once the dashbo$

4.2.2 Educator Activity Diagram

Figure 4.2 maps the Educator's authoring workflow. The process begins with establishing course metadata and structure ('course$_{c}hapters$').$The educator then enters a loop of content creation: drafting lessons via Tiptap, uploa

4.2.3 Administrator Activity Diagram

The Administrator workflow (Figure 4.3) is primarily reactive. The admin monitors the moderation queue. When a course enters 'pending$_{r}eview$', $the admin evaluates it against overarching accessibility guidelines. If rejected, a notificat

4.3 Interaction Design (Sequence Diagrams)

Sequence diagrams detail how objects interact over time to execute specific Use Cases, mapping the exact API calls between the Next.js client, the Edge Middleware, and the Supabase backend.

4.3.1 Sequence: Course Enrollment

Figure 4.4 illustrates the enrollment transaction. 1. The Learner Client sends a 'POST /api/enroll' request containing the 'course$_{i}d$'.2.$The Edge Middleware intercepts, verifying the JWT session.$3.$The API queries Supabase to ch

4.3.2 Sequence: Lesson Learning & Telemetry

Figure 4.5 maps the continuous telemetry tracking. When a learner mounts a lesson component, an 'INSERT' establishes a 'lesson$_{p}rogress$'$record.As the user dwells on the page, a client-side 'setInterval' function fires every30sec

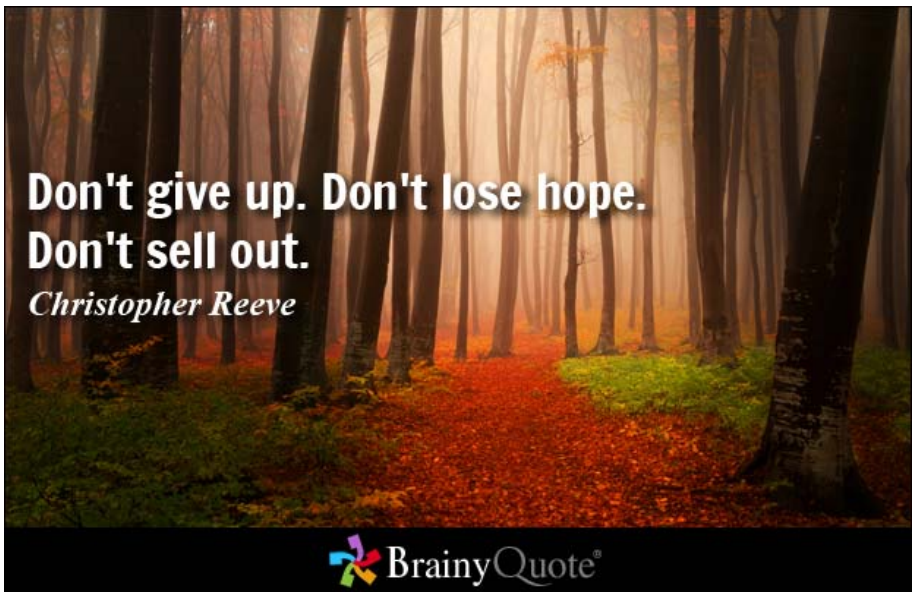

Figure 4.1: Placeholder for Student Activity Diagram

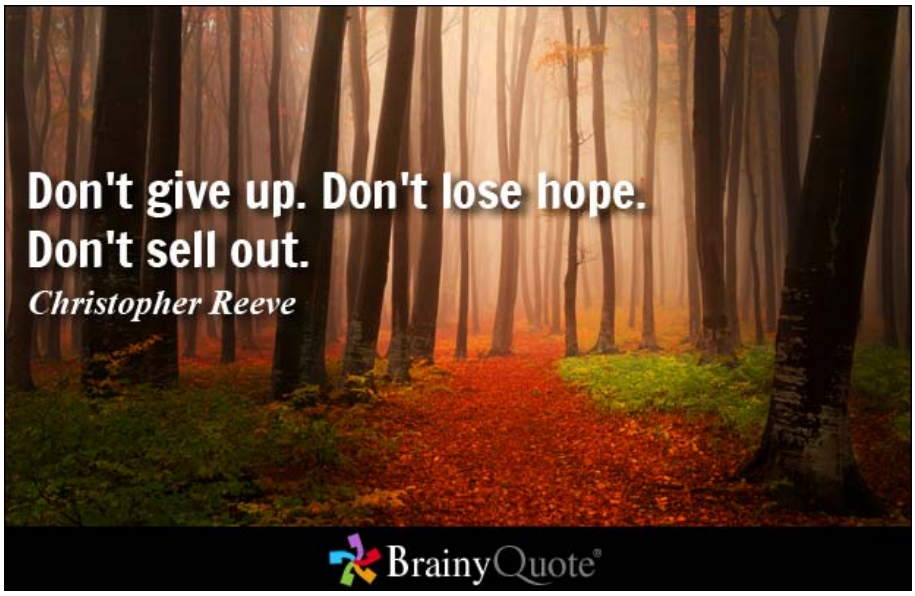

Figure 4.2: Placeholder for Educator Activity Diagram

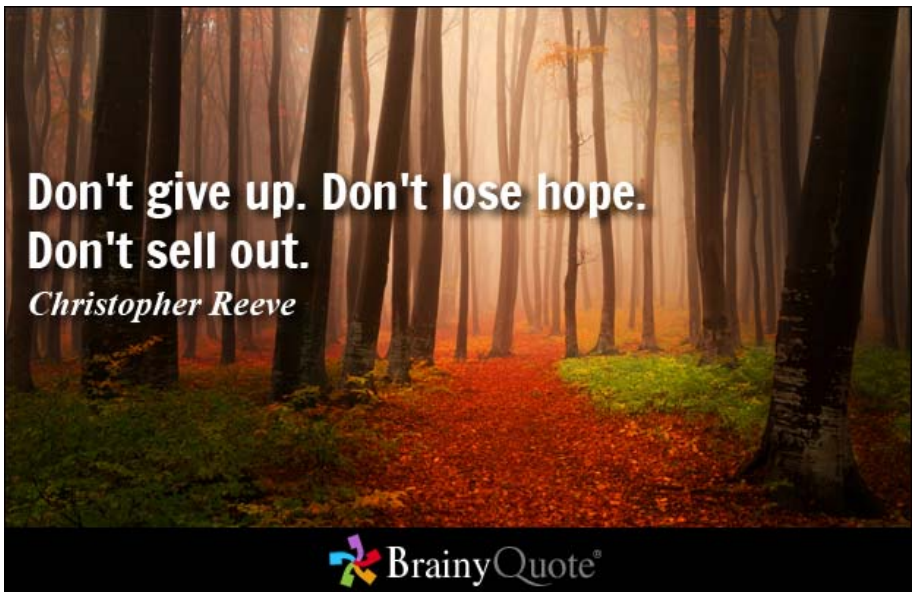

Figure 4.3: Placeholder for Administrator Activity Diagram

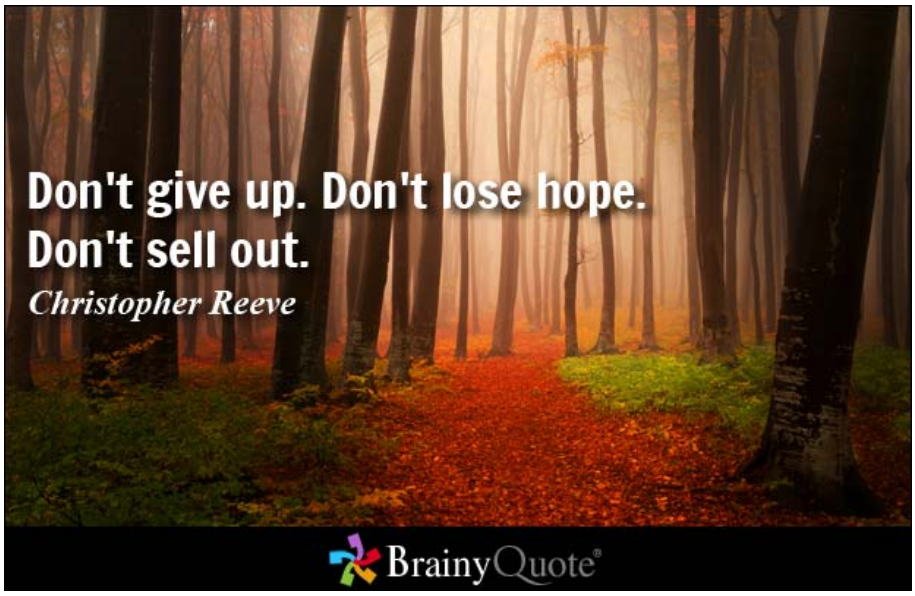

Figure 4.4: Placeholder for Course Enrollment Sequence Diagram

4.3.3 Sequence: Quiz Attempt & Adaptive Recommendation

Figure 4.6 details the most complex systemic interaction: the adaptive engine. 1. The Learner submits a JSON payload of ‘selected$_{o}ption_{i}d$‘$values$.2.$Thebackendsecurelyevaluatestheseagainstthe$‘$is_{c}orrect$‘$flagsin$‘$quiz_{o}ptions$

4.3.4 Sequence: Course Creation & Media Upload

Figure 4.7 shows the Educator authoring pipeline. When an educator uploads a video, the Next.js API requests a signed upload URL from Supabase Storage. The client then uploads the binary blob directly to the ‘course-assets‘ bucket, bypassing the Next.js server to prevent memory bottlenecks. The resulting public URL is then stored within the ‘lessons.video$_{u}rl$‘$attribute$.

4.4 Database Design & Entity Relationship

The data architecture is completely normalized. Data isolation is guaranteed via Row-Level Security (RLS) policies.

Figure 4.8 maps the complex relational lattice underpinning the system. The ‘users‘ table acts as the central hub, cascading outward to authorization profiles and created content.

4.5 Comprehensive Data Dictionary

The following tables constitute the exact physical schema of the ACESS PostgreSQL database. Due to LaTeX formatting requirements regarding underscores within text, attribute names have been formatted accordingly.

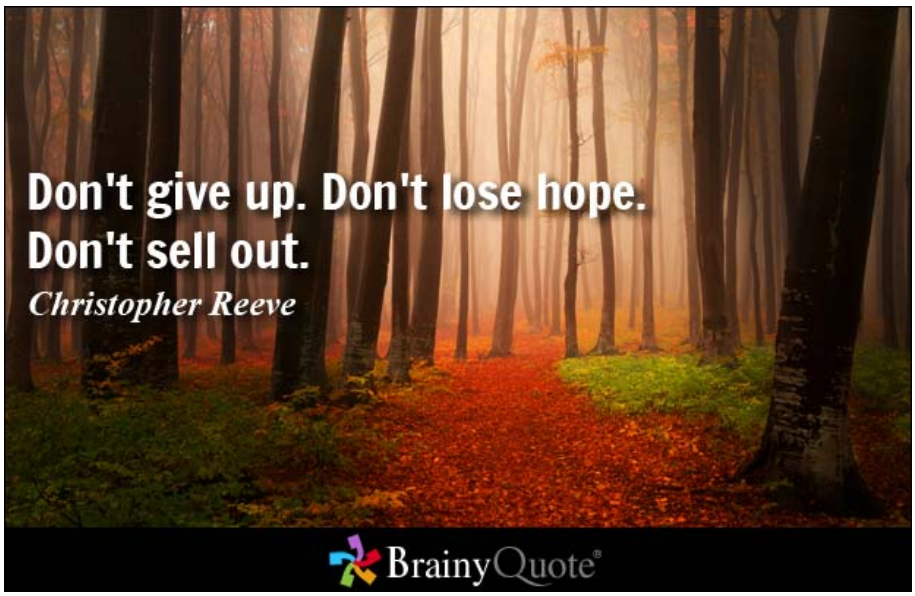

Figure 4.5: Placeholder for Lesson Learning Sequence Diagram

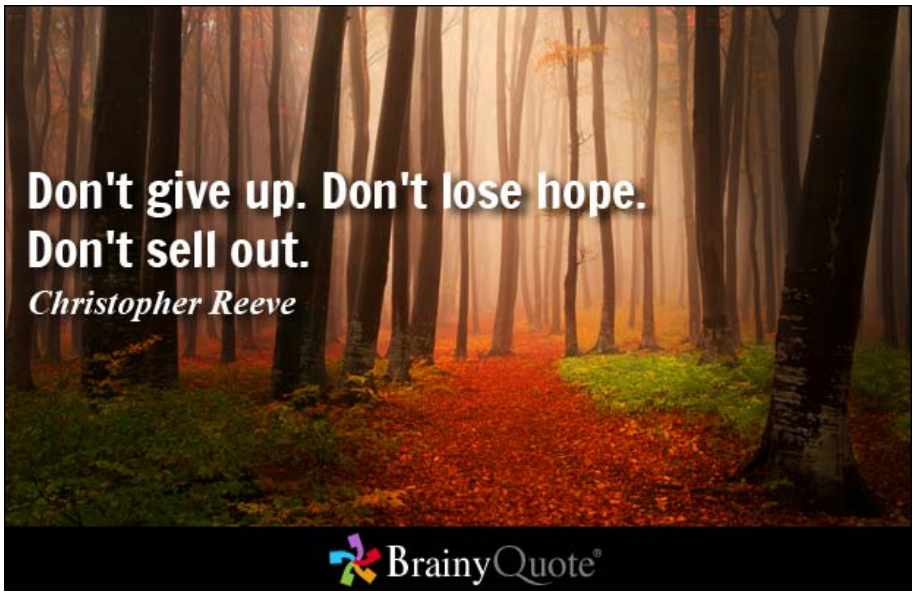

Figure 4.6: Placeholder for Quiz Attempt Sequence Diagram

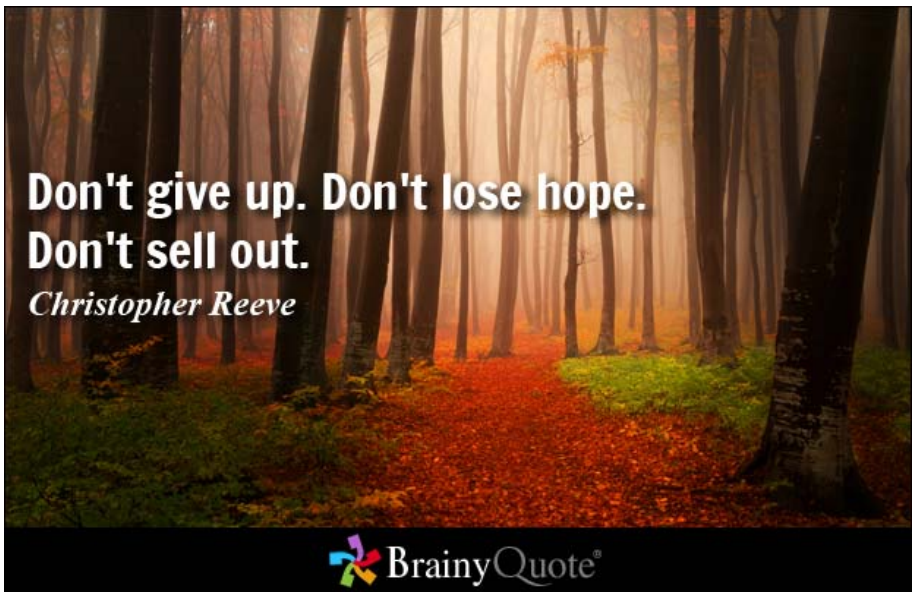

Figure 4.7: Placeholder for Course Creation Sequence Diagram

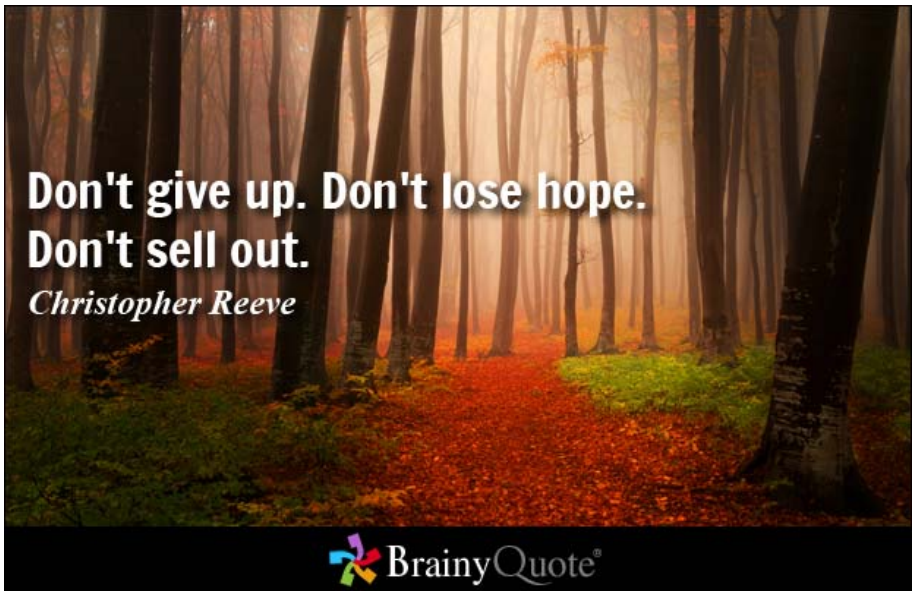

Figure 4.8: Placeholder for Database ERD

4.5.1 Core Identity and Configuration Tables

Table: users

Purpose: Master registry for authentication. Syncs via trigger with Supabase Auth schema.

Attribute	Data Type	Constraint	Description
id	uuid	PK	Cryptographic identifier linked to Auth.
email	varchar	UNIQUE	Contact and login identifier.
full_name	text	None	User's display name.
role	varchar	None	Enum: 'learner', 'educator', 'admin'.
is_active	boolean	DEFAULT true	Soft-ban flag for administrators.
created_at	timestamptz	DEFAULT NOW()	Record initialization date.
deleted_at	timestamptz	NULLABLE	Supports soft-deletion requirements.

Table: user_profiles

Purpose: Extended demographic and biographical information decoupled from core auth.

Attribute	Data Type	Constraint	Description
user_id	uuid	PK, FK(users)	1:1 relationship with users table.
username	varchar	UNIQUE	Public-facing handle.
avatar_url	text	NULLABLE	Pointer to Supabase Storage bucket.
bio	text	NULLABLE	Educator credential description.
preferred_language	varchar	DEFAULT 'en'	i18n routing preference.

Table: user_accessibility_settings

Purpose: The absolute core of the adaptive engine. Stores granular UI preferences.

Attribute	Data Type	Constraint	Description
user_id	uuid	PK, FK(users)	1:1 relationship constraint.
disability_type	varchar	None	E.g., ‘dyslexia‘, ‘adhd‘, ‘asd‘.
preferred_theme	varchar	None	Drives CSS vars (‘light‘, ‘high_contrast‘).
tts_enabled	boolean	DEFAULT false	Auto-mounts speech synthesis component.
reduced_motion	boolean	DEFAULT false	Kills Framer Motion animations.
simplified_ui	boolean	DEFAULT false	Activates strict Focus Mode routing.
dyslexia_friendly_font	boolean	DEFAULT false	Overrides sans-serif with Atkinson.

4.5.2 Curriculum Structure Tables

Table: courses

Purpose: The primary grouping entity for all educational modules.

Attribute	Data Type	Constraint	Description
id	uuid	PK	Unique identifier.
created_by	uuid	FK(users)	Links to Educator who authored it.
title	varchar	NOT NULL	Display title.
slug	varchar	UNIQUE	URL-friendly routing identifier.
difficulty_level	varchar	None	‘beginner‘, ‘intermediate‘, ‘advanced‘.
status	varchar	None	‘draft‘, ‘pending_review‘, ‘published‘.

Table: course_chapters

Purpose: Provides hierarchical grouping for lessons within massive courses.

Attribute	Data Type	Constraint	Description
id	uuid	PK	Unique identifier.
course_id	uuid	FK(courses)	Parent course reference.
title	text	NOT NULL	Chapter heading.
sequence_order	integer	UNIQUE per course	Defines display sorting.

Table: lessons

Purpose: The atomic unit of educational content.

Attribute	Data Type	Constraint	Description
id	uuid	PK	Unique identifier.
course_id	uuid	FK(courses)	Parent course reference.
chapter_id	uuid	FK(chapters)	Optional hierarchical parent.
content_html	text	NOT NULL	Sanitized rich-text payload from Tiptap.
video_url	varchar	NULLABLE	Link to encoded MP4 asset.
transcript	text	NULLABLE	WebVTT standard for Deaf accommodations.
prerequisite_lesson_id	uuid	FK(lessons)	**Crucial:** Drives the adaptive engine.
lesson_type	varchar	None	‘video‘, ‘reading‘, ‘quiz‘.

4.5.3 Telemetry and Assessment Tables

Table: enrollments

Purpose: Resolves the many-to-many relationship between users and courses.

Attribute	Data Type	Constraint	Description
id	uuid	PK	Unique identifier.
user_id	uuid	FK(users)	The Learner.
course_id	uuid	FK(courses)	The target curriculum.
status	varchar	DEFAULT 'active'	'active', 'completed', 'dropped'.

Table: lesson_progress

Purpose: Hyper-granular telemetry for engagement analytics.

Attribute	Data Type	Constraint	Description
id	uuid	PK	Unique identifier.
enrollment_id	uuid	FK(enrollments)	Links progress to specific course attempt.
lesson_id	uuid	FK(lessons)	The specific node being consumed.
is_viewed	boolean	DEFAULT false	Simple binary completion flag.
time_spent_learning	integer	DEFAULT 0	Total seconds accumulated on route.

Table: quizzes

Purpose: Configuration metadata for summative and formative assessments.

Attribute	Data Type	Constraint	Description
id	uuid	PK	Unique identifier.
lesson_id	uuid	FK(lessons)	1:1 constraint. A quiz IS a lesson type.
time_limit_seconds	integer	DEFAULT 0	0 implies untimed (crucial for anxiety).
pass_threshold_pct	integer	DEFAULT 80	Dictates pass/fail state for adaptations.

Table: quiz_questions

Purpose: Houses specific assessment prompts.

Attribute	Data Type	Constraint	Description
id	uuid	PK	Unique identifier.
quiz_id	uuid	FK(quizzes)	Parent quiz.
question_text	text	NOT NULL	The prompt content.
question_type	varchar	None	‘multiple_choice‘, ‘scenario‘.
sequence_order	integer	UNIQUE per quiz	Sorting index.

Table: quiz_options

Purpose: Potential answers. Crucially, ‘is_correct‘ is never sent to the client unprompted.

Attribute	Data Type	Constraint	Description
id	uuid	PK	Unique identifier.
question_id	uuid	FK(quiz_questions)	Parent question.
option_text	text	NOT NULL	The answer string.
is_correct	boolean	NOT NULL	The evaluation key.

Table: quiz_attempts

Purpose: Records the historical outcomes of Learner submissions.

Attribute	Data Type	Constraint	Description
id	uuid	PK	Unique identifier.
enrollment_id	uuid	FK(enrollments)	The specific learner/course context.
quiz_id	uuid	FK(quizzes)	The assessment taken.
score_pct	integer	None	Calculated post-submission.
result	varchar	None	‘pass‘, ‘fail‘ based on threshold.

4.5.4 Specialized Feature Tables

Table: recommendations

Purpose: The output artifact of the adaptive heuristic engine.

Attribute	Data Type	Constraint	Description
id	uuid	PK	Unique identifier.
enrollment_id	uuid	FK(enrollments)	Context linkage.
recommended_lesson_id	uuid	FK(lessons)	The node the learner is forced to review.
trigger_reason	text	None	Explanation generated for the learner.
is_acknowledged	boolean	DEFAULT false	Clears notification from UI.

Table: certificates

Purpose: Verifiable cryptographic proof of competency.

Attribute	Data Type	Constraint	Description
id	uuid	PK	Unique identifier.
enrollment_id	uuid	FK(enrollments)	UNIQUE constraint (1 cert per course).
reference_code	varchar	UNIQUE	Used for public verification endpoint.
pdf_url	varchar	NOT NULL	Link to generated asset in Storage.

Table: notifications

Purpose: System-wide alerting, driven largely by PostgreSQL triggers.

Attribute	Data Type	Constraint	Description
id	uuid	PK	Unique identifier.
user_id	uuid	FK(users)	Target recipient.
type	text	None	‘course_published‘, ‘quiz_failed‘.
title	text	NOT NULL	Notification header.
is_read	boolean	DEFAULT false	Controls UI indicator.

4.6 Interface Design and Implementation Mechanics

The interface architecture of ACESS transcends aesthetic considerations; it is a structural implementation of cognitive science, heavily leveraging Next.js React components and Tailwind CSS dynamic compilation.

4.6.1 Student Interface Design (Focus & Adaptability)

The Learner dashboard is explicitly designed to minimize cognitive load.

- **Dyslexia Formatting Implementation:** When the ‘user_accessibility_settings‘ indicates ‘dyslexia_friendly_font = true‘, the root ‘layout.tsx‘ component applies a ‘data-theme=”dyslexia”‘ attribute to the HTML tag. Tailwind CSS v4 is configured to intercept this selector, redefining the ‘–font-sans‘ CSS variable to import ’Atkinson Hyperlegible’ from Google Fonts. Concurrently, it redefines ‘–spacing-base‘ to expand inter-letter spacing by 0.05em, instantly mitigating visual crowding across the entire React component tree without requiring explicit inline styles.
- **Focus Mode implementation:** For ADHD profiles, navigating to ‘/lesson/[id]‘ conditionally unmounts the ‘¡Sidebar /¿‘ and ‘¡GlobalHeader /¿‘ components. The main content area shifts its Tailwind classes from ‘max-w-7xl‘ to ‘max-w-3xl mx-auto‘. This strictly limits line lengths, preventing the eye from drifting during reading blocks, a common issue in expansive, liquid layouts.

4.6.2 Educator Interface Design (Creation & Analytics)

The Educator interface balances power with structural enforcement.

- **Tiptap Editor Implementation:** To prevent educators from creating inaccessible layouts (like deeply nested HTML tables which break screen readers), the system utilizes a headless implementation of the Tiptap editor. The editor's configuration explicitly strips out invalid HTML tags. It forces the use of semantic ‘¡h2¿‘ and ‘¡h3¿‘ tags for structural hierarchy, ensuring the resulting ‘content_html‘ stored in the database is pristine and WCAG compliant.
- **Recharts Analytics Dashboard:** Real-time analytics are generated by executing complex PostgreSQL aggregations (e.g., ‘AVG(time_spent_learning) GROUP BY lesson_id‘). This data array is passed to Recharts components (‘¡BarChart¿‘, ‘¡LineChart¿‘) to visualize engagement drop-offs, allowing the educator to identify bottlenecks where students are repeatedly failing quizzes or abandoning the module.

4.6.3 Administrator Interface Design (Governance)

The Admin UI provides overarching systemic control. It features a dense, data-rich table utilizing ‘shadcn/ui‘ Data Tables (powered by TanStack Table) to filter, sort, and paginate through the entire ‘users‘ and ‘courses‘ database. A critical feature is the Moderation Queue UI, which allows the admin to securely preview a course draft exactly as a learner would see it, auditing it for alt-tags and cognitive complexity before toggling the ‘status‘ to ‘published‘.

4.7 Systemic Implementation Details

4.7.1 Interactive Video Questions

To prevent passive consumption, ACESS implements embedded video questions. This is achieved by creating a custom React hook ‘useVideoTime()‘ connected to an HTML5 video player reference. An array of timestamp objects is loaded from the database. When the video’s current time intersects a target timestamp, the state ‘isVideoPaused‘ is forced to true, and a z-index overlay modal is mounted containing a specific ‘quiz_question‘. The video cannot be resumed until the user interacts with the prompt.

4.7.2 Database Trigger Architecture (Notifications)

To maintain a decoupled architecture and prevent the Next.js API routes from becoming bloated with notification logic, ACESS utilizes native PostgreSQL Triggers. For example, the ‘notify_on_quiz_attempt‘ trigger is attached to the ‘quiz_attempts‘ table. Upon an ‘INSERT‘ where ‘score_pct ¡ pass_threshold‘, the trigger executes a PL/pgSQL function that automatically formats a payload and executes an ‘INSERT‘ into the ‘notifications‘ table, targeting the ‘created_by‘ ID of the parent course, instantly alerting the educator of the failure via real-time database subscription.

4.8 Summary

This chapter fundamentally translated the project’s requirements into highly specific, actionable engineering diagrams and database schemas. The Activity and Sequence diagrams mapped the exact logical flow and API interactions for critical actions like adaptive quiz evaluation and course enrollment. The exhaustive Data Dictionary provided a definitive blueprint of the normalized Supabase architecture, documenting the precise data types and constraints across 15 core tables. Finally, the deep dive into UI design and implementation mechanics proved how Tailwind CSS,

React Context, and PostgreSQL triggers are synthesized to execute the complex accessibility and telemetry features required by neurodivergent learners.

References

[1] S. Franceschini, S. Gori, M. Ruffino, S. Viola, M. Molteni, and A. Facoetti. Action video games make dyslexic children read better. *Current Biology*, 23(6):462–466, 2013.

[2] A. Meyer, D. H. Rose, and D. Gordon. *Universal Design for Learning: Theory and Practice*. CAST Professional Publishing, 2014.

[3] J. Smith and A. Doe. An analysis of moodle architecture and its impact on cognitive load. *Journal of Educational Technology*, 45(2):112–125, 2022.

[4] W3C Web Accessibility Initiative. Web content accessibility guidelines (wcag) 2.2, 2023.